What does it mean when we say that the `must` clause accepts a set of multiple queries as an array? It means we can add queries to the `must` clause to make it much more sophisticated. As an example, the following code includes a `must` clause with two leaf queries: a `match` query searching for the `author` and a `match_phrase` query searching for a phrase.

Listing 2.20 A `must` clause with multiple leaf queries

```
GET books/_search
{
  "query": {
    "bool": {
      "must": [{
          "match": {
            "author": "Joshua Bloch"
          }
        },
        {
          "match_phrase": {
            "synopsis": "best Java programming books"
          }
        }]
      }
  }
}
```

must query with two leaf queries

match query finding books authored by Joshua

A second query searching for a phrase in a field

Let's move on and add a `must_not` clause, which creates a negation criteria.

2.5.3 The must_not clause

Let's improve our criteria. We shouldn't fetch Joshua's books if the ratings are below 4.7. To satisfy this condition, we use a `must_not` clause with a `range` query that sets the rating to less than (`lt`) 4.7. The following listing demonstrates the `must_not` clause, along with the `must` clause from the original query.

Listing 2.21 A `bool` query with `must` and `must_not` clauses

```
GET books/_search
{
  "query": {
    "bool": {
      "must": [{ "match": { "author": "Joshua" } }],
     "must_not": [{ "range": { "amazon_rating": { "lt": 4.7}}}]
    }
  }
}
```

must clause that searches for Joshua's books

must_not clause with a range query that excludes books with lower ratings

This query results in only one book, *Effective Java*. Joshua Bloch's other book (*Java Concurrency in Practice*) is dropped from the list because it does not fit the `must_not` criterion (its rating is 4.3, which is less than the prerequisite of 4.7).